



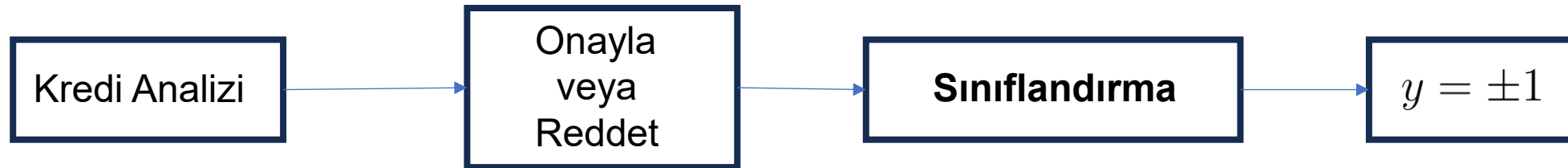
Institute for Data Science & Artificial Intelligence

Sınıflandırma, Kümeleme & Regülerize edilmiş regresyon

Sınıflandırma (Classification)

Tanım (Sınıflandırma)

Makine öğrenmesinde **sınıflandırma**, verilen bir veri örneğinin özelliklerine (özniteliklerine) bakarak onu önceden tanımlanmış sınıflardan birine atama problemidir. Amaç, eğitim verilerinden öğrenilen bir model kullanarak yeni ve daha önce görülmemiş verilerin hangi sınıfa ait olduğunu doğru şekilde tahmin etmektir.

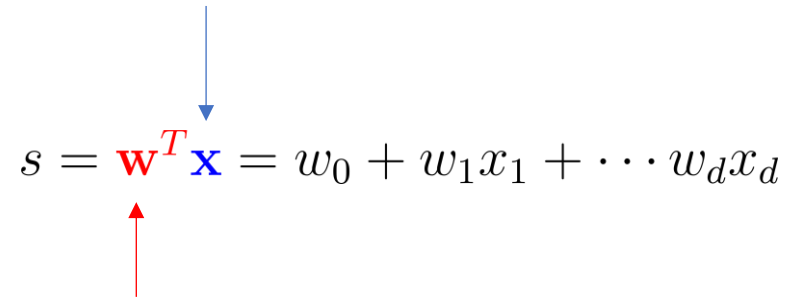


Sınıflandırma için bir sürü model türü kullanılabilir (doğrusal, doğrusal-olmayan)

- Doğrusal model (linear model) belki de **en temel** modeldir.
- Birçok makine öğrenmesi probleminde denenecek **ilk model** doğrusal modeldir.

Doğrusal model bazlı sınıflandırma için önce **doğrusal sinyal** nedir ona bakalım...

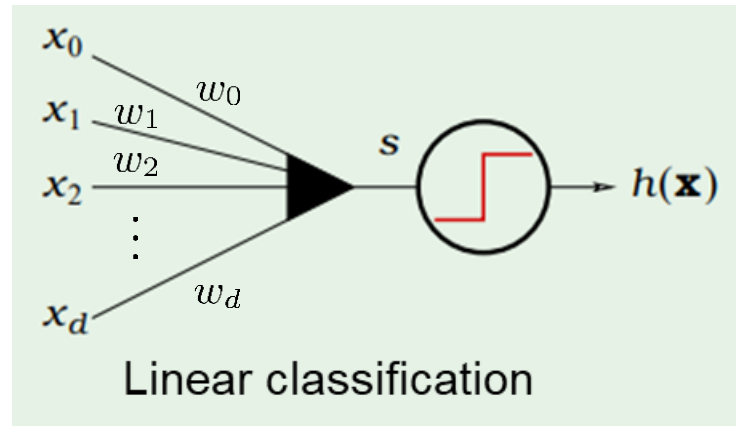
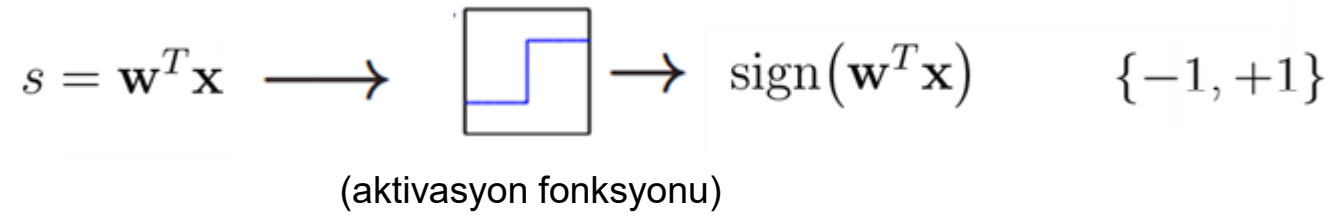
x 'e göre doğrusal : çizgi/hiperdüzlem ayıracını verir (the line/hyperplane separator)


$$s = \mathbf{w}^T \mathbf{x} = w_0 + w_1 x_1 + \cdots w_d x_d$$

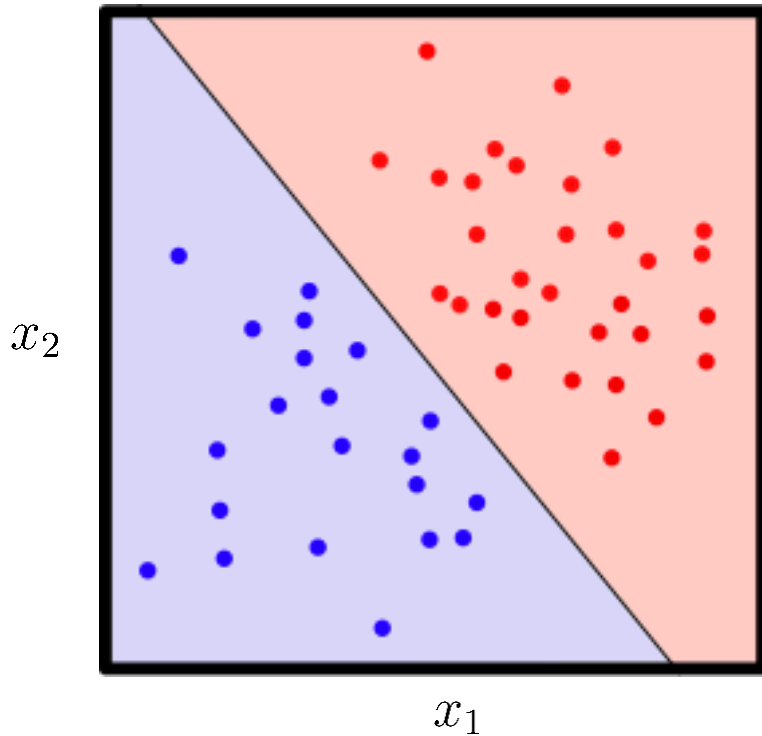
w 'e göre doğrusal olma algoritmanın “kolay” çalışmasını sağlar.

Not: x genişletilmiş (augmented) vektördür: $x \in \{1\} \times \mathbb{R}^d$.

Doğrusal sinyal (2)

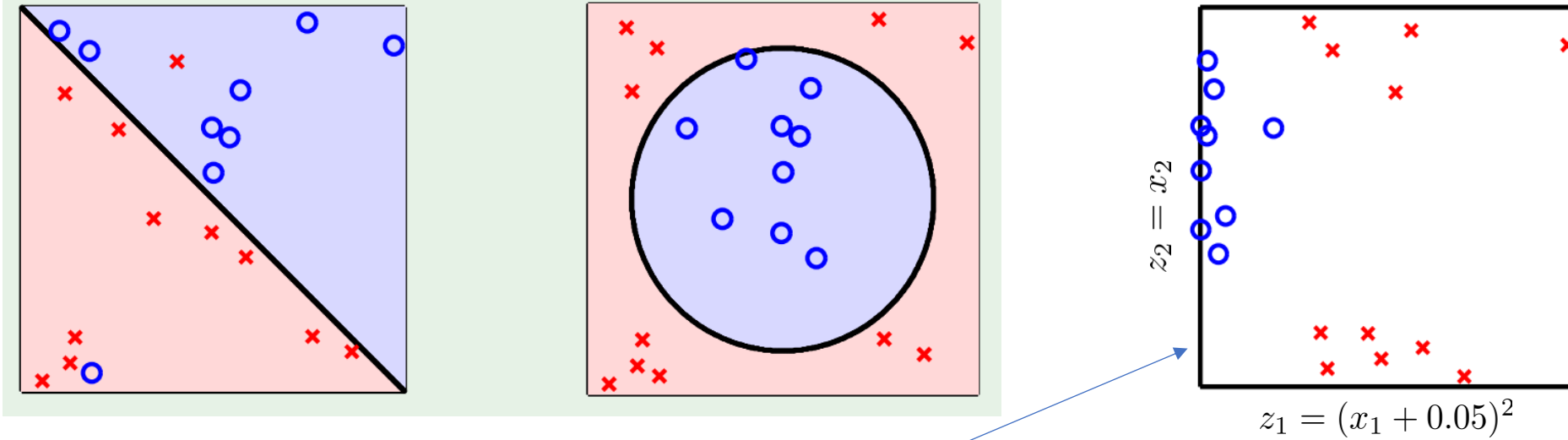


Doğrusal ayrılabilir durum:



- ❑ Eğer sınıf verisi doğrusal olarak ayrılabiliyorsa, o zaman “**Perceptron Learning Algorithm (PLA)**” kullanılabilir.
- ❑ PLA detayları için paylaşılan ML notlarına bakabilirsiniz.

Doğrusal ayrılamayan durum (non-linearly separable case):



Bu durumda, duruma göre aşağıdaki seçenekler mevcuttur:

- Sınıflandırma verisi “**yaklaşık**” doğrusal olarak ayrılabilirse, o zaman “**Pocket**” algoritması kullanılabilir.
- Sınıflandırma verisi “**görünüşte**” doğrusal olarak ayrılmıyor olabilir ama doğrusal olmayan transformasyonlar (nonlinear transformations) uygulanıp yeni düzlemde doğrusal olarak ayrılabilir.
- Sınıflandırma verisi ciddi anlamda doğrusal olarak ayrılmıyorsa o zaman ileri sınıflandırma teknikleri kullanılabilir: derin öğrenme, SVM, XGBoost, vs.

El yazısı rakam verisi (1)

Her bir rakam 16 x 16 boyutunda bir görüntüdür.

0	2	3	8	0	7	3	8	5	7
2	1	7	9	7	1	2	8	4	5
4	8	1	9	0	1	8	8	9	4
7	6	1	8	6	4	1	5	6	0
7	5	9	2	6	5	8	1	9	7
3	6	8	1	7	9	6	6	9	1
6	7	5	7	8	6	3	4	8	5
0	1	4	6	4	6	0	2	4	3

Fig: ABD Posta Servisi (US Postal Service) posta kodu veritabanından el yazısı rakam örnekleri

0	2	3	8	0	7	3	8	5	7
2	1	7	9	7	1	2	8	4	5
4	8	1	9	0	1	8	8	9	4
7	6	1	8	6	4	1	5	6	0
7	5	9	2	6	5	8	1	9	7
3	6	8	1	7	9	6	6	4	1
6	7	5	7	8	6	3	4	8	5
0	1	4	6	4	6	0	2	4	3

Her bir rakam 16 x 16 boyutunda bir görüntüdür.

Detaylı özellikler (features):

6

$$\begin{bmatrix} -1 & -1 & -1 & -1 & -1 & -1 & -1 & -0.63 & 0.86 & -0.17 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -0.99 & 0.3 & 1 & 0.31 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 \\ -1 & -1 & -0.41 & 1 & 0.99 & -0.57 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -0.68 & 0.83 & 1 & 0.56 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -0.94 & 0.54 \\ 1 & 0.78 & -0.72 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & 0.1 & 1 & 0.92 & -0.44 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -0.26 & 0.95 & 1 & -0.16 & -1 & -1 \\ -1 & -0.99 & -0.71 & -0.83 & -1 & -1 & -1 & -1 & -1 & -0.8 & 0.91 & 1 & 0.3 & -0.96 & -1 & -1 & -0.55 & 0.49 & 1 & 0.88 & 0.09 & -1 & -1 & -1 & -1 & 0.28 & 1 & 0.88 & -0.8 & -1 \\ -0.9 & 0.14 & 0.97 & 1 & 1 & 1 & 0.99 & -0.74 & -1 & -1 & -0.95 & 0.84 & 1 & 0.32 & -1 & -1 & 0.35 & 1 & 0.65 & -0.10 & -0.18 & 1 & 0.98 & -0.72 & -1 & -1 & -0.63 & 1 & 1 \\ 0.07 & -0.92 & 0.11 & 0.96 & 0.30 & -0.88 & -1 & -0.07 & 1 & 0.64 & -0.99 & -1 & -1 & -0.67 & 1 & 1 & 0.75 & 0.34 & 1 & 0.70 & -0.94 & -1 & -1 & 0.54 & 1 & 0.02 & -1 & -1 \\ -1 & -0.90 & 0.79 & 1 & 1 & 1 & 1 & 0.53 & 0.18 & 0.81 & 0.83 & 0.97 & 0.86 & -0.63 & -1 & -1 & -1 & -1 & -0.45 & 0.82 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0.13 & -1 & -1 & -1 & -1 & -1 \\ -1 & -0.48 & 0.81 & 1 & 1 & 1 & 1 & 1 & 1 & 0.21 & -0.94 & -1 & -1 & -1 & -1 & -1 & -1 & -0.97 & -0.42 & 0.30 & 0.82 & 1 & 0.48 & -0.47 & -0.99 & -1 & -1 & -1 & -1 \end{bmatrix}$$
$$\mathbf{x} = (1, x_1, \dots, x_{256}) \quad \leftarrow \text{girdi (feature)}$$
$$\mathbf{w} = (w_0, w_1, \dots, w_{256}) \leftarrow \text{doğrusal model parametre vektörü}$$

Şimdi daha basit bir problem oluşturalım: iki rakam için ikili (binary) sınıflandırmayı düşünelim sadece: 1 ve 5...

0	2	3	8	0	7	3	8	5	7
2	1	7	9	7	1	2	8	4	5
4	8	1	9	0	1	8	8	9	4
7	6	1	8	6	4	1	5	6	0
7	5	9	2	6	5	8	1	9	7
3	6	8	1	7	9	6	6	9	1
6	7	5	7	8	6	3	4	8	5
0	1	4	6	4	6	0	2	4	3

Öznitelik (feature) oluşturma: Orijinal “detaylı” öznitelik vektörü yerine, iki öznitelik düşünebiliriz bu problem için: yoğunluk ve simetri.

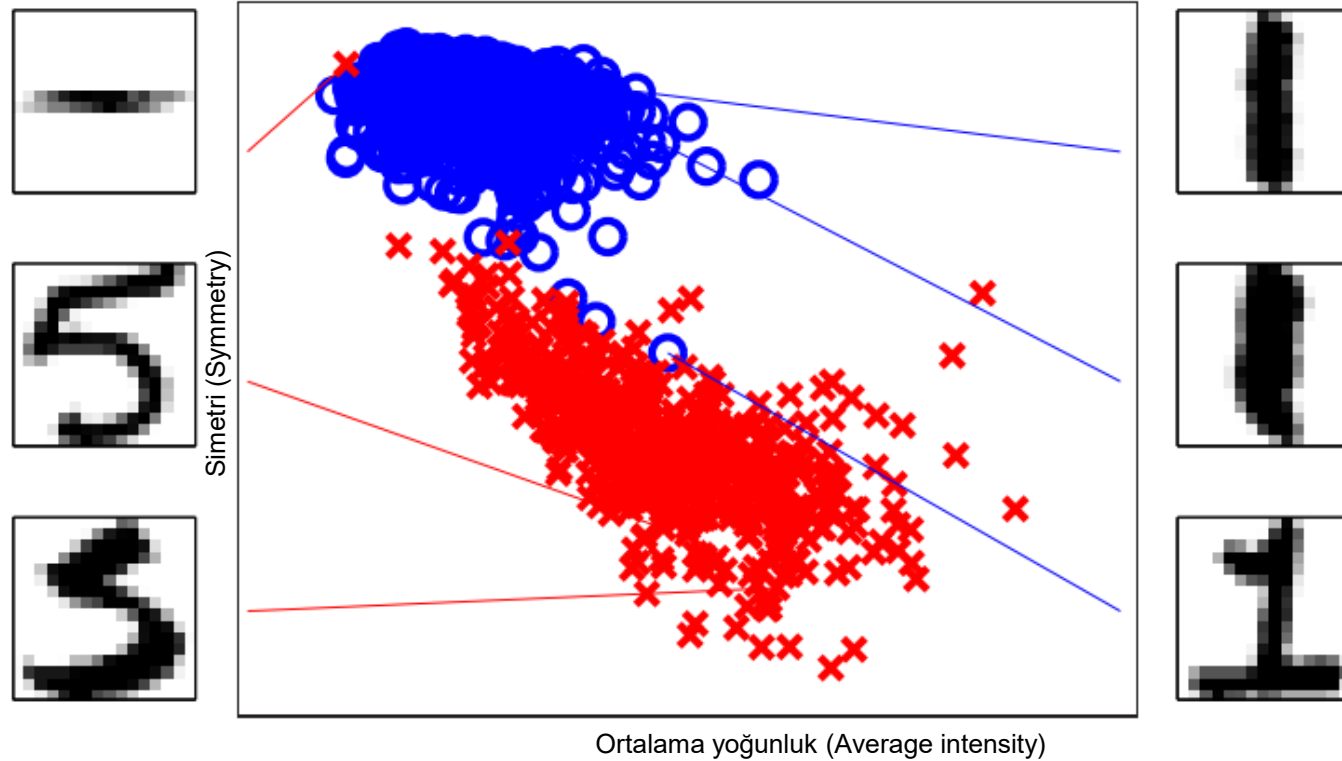
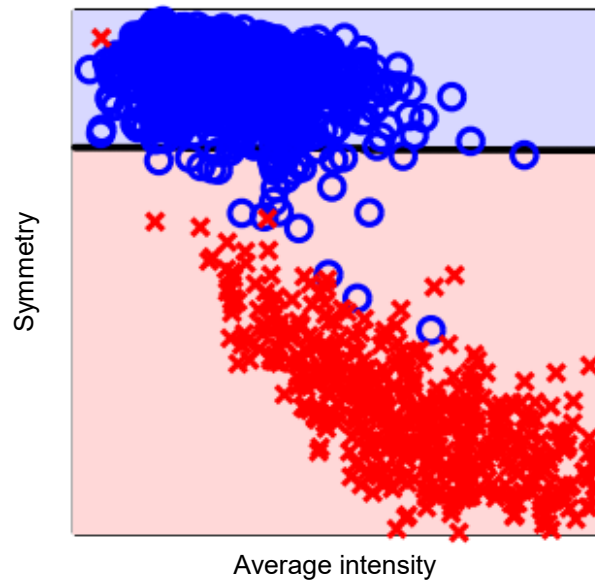
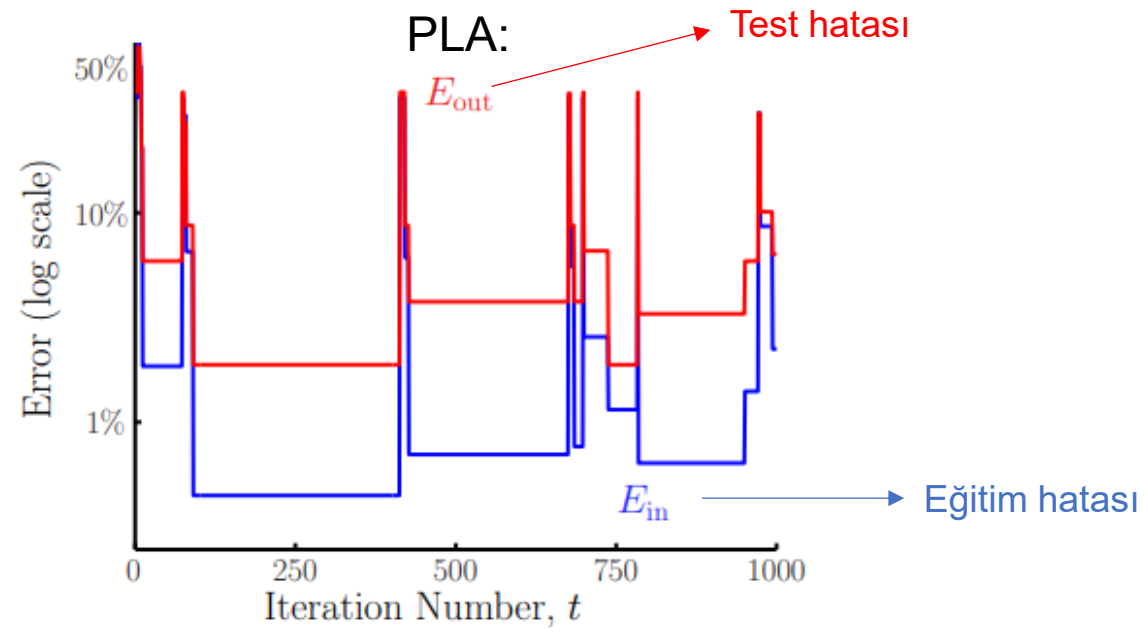


Fig: US Postal Service (USPS) Zip Code veritabanındaki tüm 1 ve 5 rakamlarının, iki basitleştirilmiş özellik kullanılarak çizilmiş grafiği.

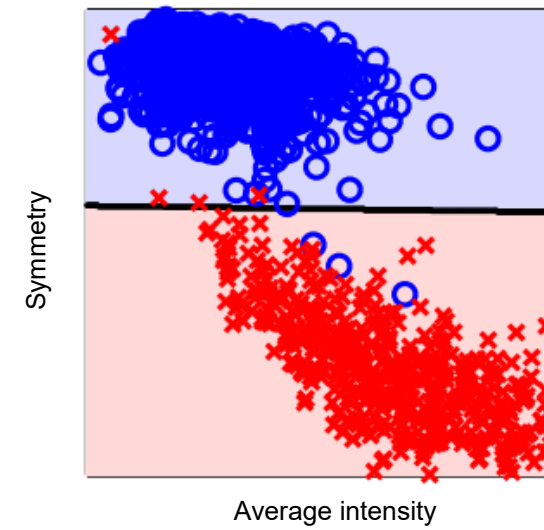
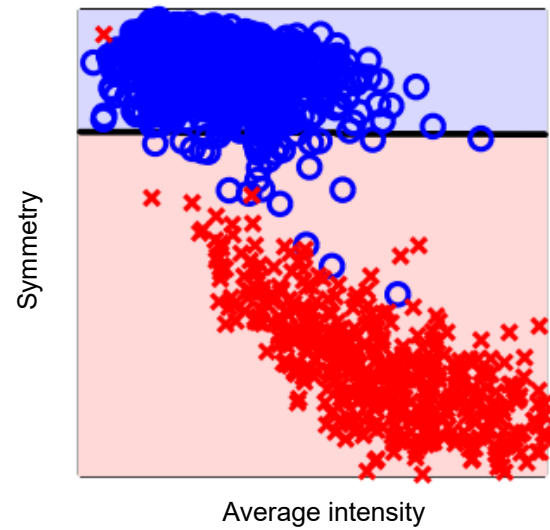
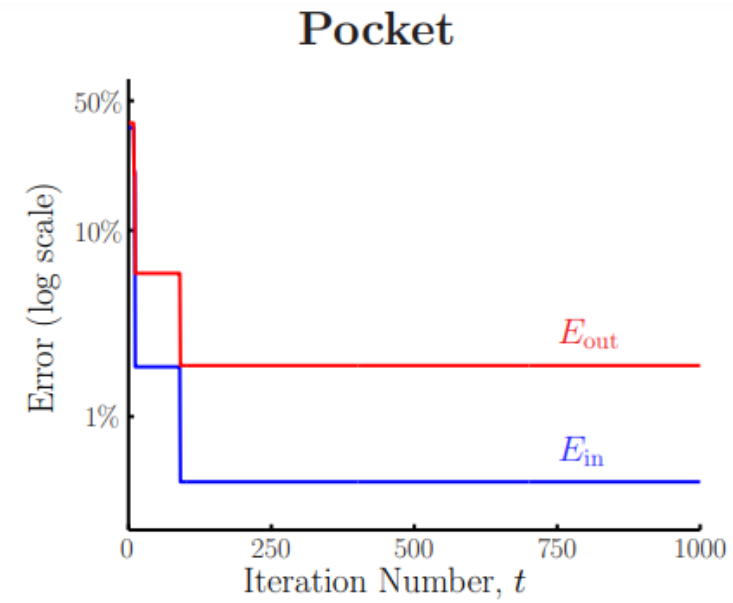
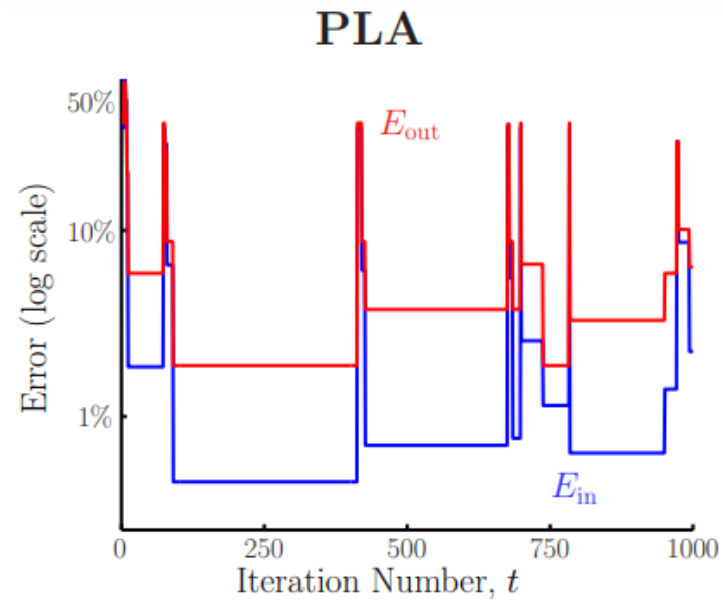
$$\mathbf{x} = (1, x_1, x_2) \quad \leftarrow \text{girdi}$$

$$\mathbf{w} = (w_0, w_1, w_2) \quad \leftarrow \text{doğrusal model parametre vektörü}$$

El yazısı rakam verisi (5)



El yazısı rakam verisi (6)



İkili (binary) sınıflandırma'ya karşı çok sınıflı (multi-class) sınıflandırma



Kümeleme (Clustering)

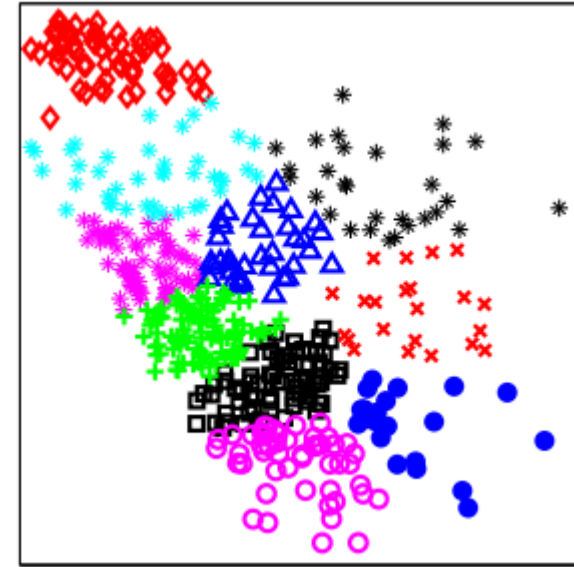
Bir **küme** (**cluster**), $\mathcal{S}$ seti içinde yer alan ve birbirine benzer özellikler gösteren noktalardan oluşan bir topluluktur.

Bir “**k-kümeleme**” (k-clustering), verinin $\mathcal{D}$, k adet alt kümeye $\mathcal{S}_1, \mathcal{S}_2, \dots, \mathcal{S}_k$ ayrılmasıdır; öyle ki

$$\cup_{j=1}^k \mathcal{S}_j = \mathcal{D}$$

$$\mathcal{S}_i \cap \mathcal{S}_j = \emptyset \quad \text{for } i \neq j$$

Her kümenin bir merkezi, μ_j , vardır.

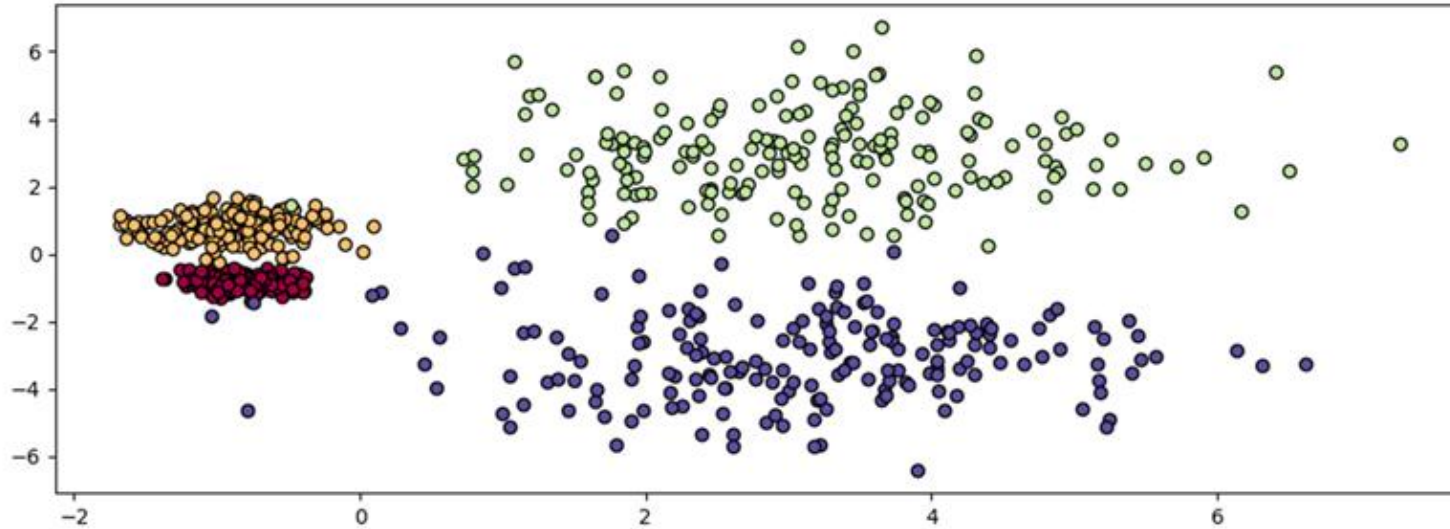


Şekil: Rakam (el-yazılı) verilerinden seçilen 500 örneğin on kümeye ayrılarak kümelenmesi.

Kümelemeyi iyi yapan özellikler nedir? (1)

Temelde, kümelerin iki iyi özelliği sağlaması gerekir:

1. Bir küme içindeki noktalar birbirine **yakın** olmalıdır.
 2. Farklı kümelerdeki noktalar birbirinden **uzak** olmalıdır.
- (1) “**küme içi yayılım**”ın (“**within cluster spread**”) küçük olması gerektiğini ifade eder.
 - (2) “**kümeler arası yayılım**”ın (“**between cluster spread**”) büyük olması gerektiğini ifade eder.



Not: (1)'i sağlamaya odaklanacağız; çünkü (1) sağlandığında (2) de aşağı yukarı kendiliğinden sağlanacaktır.

Bir küme içindeki noktalar benzer olmalıdır (birbirine ve merkeze yakın olmalıdır).

j kümesindeki hata:

$$E_j = \sum_{\mathbf{x}^n \in \mathcal{S}_j} \|\mathbf{x}^n - \boldsymbol{\mu}_j\|^2 \quad (j\text{'inci küme için yayılım})$$

k-ortalama (k-means) kümeleme hatası:

$$\begin{aligned} E_{\text{in}}(\mathcal{S}_1, \dots, \mathcal{S}_k; \boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_k) &= \sum_{j=1}^k E_j \\ &= \sum_{n=1}^N \|\mathbf{x}^n - \boldsymbol{\mu}(\mathbf{x}^n)\|^2 \end{aligned}$$

$\boldsymbol{\mu}(\mathbf{x}^n)$, $\mathbf{x}^n$ 'in ait olduğu kümenin merkezidir.

Dolayısıyla, E_{in} mümkün olduğunca küçükse kümeleme “iyi” olacaktır.

Kümeleme çıktıları şunlardır:

- $\mathcal{S}_1, \dots, \mathcal{S}_k$ kümeleri
- küme merkezleri: $\mu_1, \dots, \mu_k$

Not: Kümeleme zor bir optimizasyon problemidir; ancak problemi yaklaşık olarak çözmek için sezgisel bir algoritma sunacağız.

$E_{\text{in}}(\mathcal{S}_1, \dots, \mathcal{S}_k; \mu_1, \dots, \mu_k)$ ifadesini en aza indirmek için $\mathcal{S}_1, \dots, \mathcal{S}_k$ ve $\mu_1, \dots, \mu_k$ seçmemiz gerekir.

Gözlem:

- Eğer merkezler μ_j biliniyorsa, kümeleri seçmek kolaydır: μ_j 'ye en yakın tüm noktaları $\mathcal{S}_j$ 'ye ekleyin.
- Eğer kümeler $\mathcal{S}_j$ biliniyorsa, merkezleri seçmek kolaydır: merkez μ_j , $\mathcal{S}_j$ kümesinin ağırlık merkezidir:

$$\mu_j = \frac{1}{|\mathcal{S}_j|} \sum_{\mathbf{x}^n \in \mathcal{S}_j} \mathbf{x}^n$$

$$E_{\text{in}}(\mathcal{S}_1, \dots, \mathcal{S}_k; \boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_k) = \sum_{n=1}^N \|\mathbf{x}^n - \boldsymbol{\mu}(\mathbf{x}^n)\|^2$$

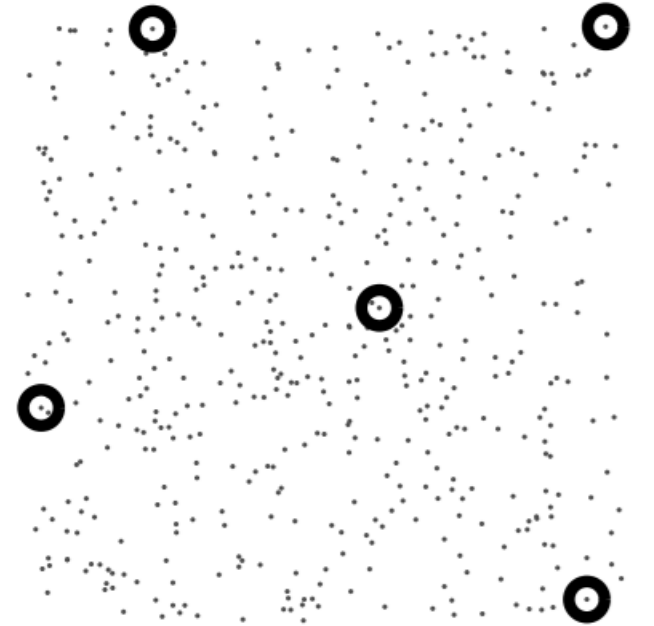
1. **Başlatma:** Birbirinden iyi ayrılmış merkezler $\boldsymbol{\mu}_j$ seçin.
2. **Güncelle:** $\mathcal{S}_j$ 'yi, $\boldsymbol{\mu}_j$ 'ye en yakın tüm noktalar olacak şekilde güncelleyin:

$$\mathcal{S}_j \leftarrow \left\{ \mathbf{x}^n : \|\mathbf{x}^n - \boldsymbol{\mu}_j\| \leq \|\mathbf{x}^n - \boldsymbol{\mu}_l\| \text{ for } l = 1, \dots, k \right\}$$

3. $\boldsymbol{\mu}_j$ 'yi $\mathcal{S}_j$ kümesinin ağırlık merkezi (centroid) olacak şekilde güncelleyin:

$$\boldsymbol{\mu}_j \leftarrow \frac{1}{|\mathcal{S}_j|} \sum_{\mathbf{x}^n \in \mathcal{S}_j} \mathbf{x}^n$$

4. E_{in} istenilen azalmayı sağlayana kadar 2 ve 3. adımları tekrarlayın.



$$E_{\text{in}}(\mathcal{S}_1, \dots, \mathcal{S}_k; \boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_k) = \sum_{n=1}^N \|\mathbf{x}^n - \boldsymbol{\mu}(\mathbf{x}^n)\|^2$$

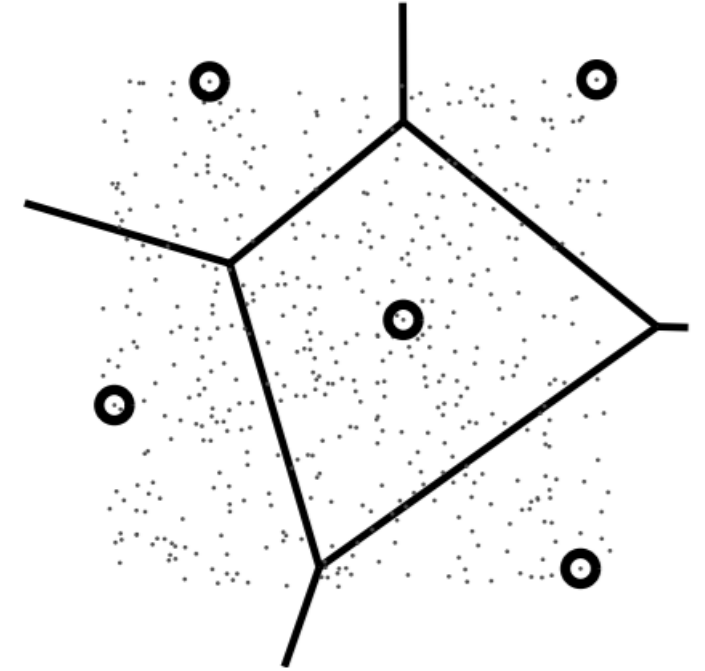
1. **Başlatma:** Birbirinden iyi ayrılmış merkezler $\boldsymbol{\mu}_j$ seçin.
2. **Güncelle:** $\mathcal{S}_j$ 'yi, $\boldsymbol{\mu}_j$ 'ye en yakın tüm noktalar olacak şekilde güncelleyin:

$$\mathcal{S}_j \leftarrow \left\{ \mathbf{x}^n : \|\mathbf{x}^n - \boldsymbol{\mu}_j\| \leq \|\mathbf{x}^n - \boldsymbol{\mu}_l\| \text{ for } l = 1, \dots, k \right\}$$

3. $\boldsymbol{\mu}_j$ 'yi $\mathcal{S}_j$ kümesinin ağırlık merkezi (centroid) olacak şekilde güncelleyin:

$$\boldsymbol{\mu}_j \leftarrow \frac{1}{|\mathcal{S}_j|} \sum_{\mathbf{x}^n \in \mathcal{S}_j} \mathbf{x}^n$$

4. E_{in} istenilen azalmayı sağlayana kadar 2 ve 3. adımları tekrarlayın.



$$E_{\text{in}}(\mathcal{S}_1, \dots, \mathcal{S}_k; \boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_k) = \sum_{n=1}^N \|\mathbf{x}^n - \boldsymbol{\mu}(\mathbf{x}^n)\|^2$$

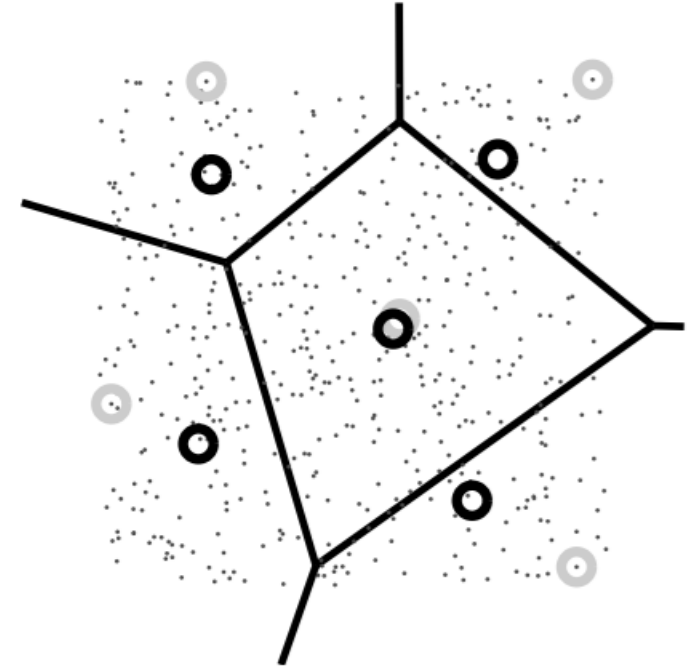
1. **Başlatma:** Birbirinden iyi ayrılmış merkezler $\boldsymbol{\mu}_j$ seçin.
2. **Güncelle:** $\mathcal{S}_j$ 'yi, $\boldsymbol{\mu}_j$ 'ye en yakın tüm noktalar olacak şekilde güncelleyin:

$$\mathcal{S}_j \leftarrow \left\{ \mathbf{x}^n : \|\mathbf{x}^n - \boldsymbol{\mu}_j\| \leq \|\mathbf{x}^n - \boldsymbol{\mu}_l\| \text{ for } l = 1, \dots, k \right\}$$

3. $\boldsymbol{\mu}_j$ 'yi $\mathcal{S}_j$ kümesinin ağırlık merkezi (centroid) olacak şekilde güncelleyin:

$$\boldsymbol{\mu}_j \leftarrow \frac{1}{|\mathcal{S}_j|} \sum_{\mathbf{x}^n \in \mathcal{S}_j} \mathbf{x}^n$$

4. E_{in} istenilen azalmayı sağlayana kadar 2 ve 3. adımları tekrarlayın.



$$E_{\text{in}}(\mathcal{S}_1, \dots, \mathcal{S}_k; \boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_k) = \sum_{n=1}^N \|\mathbf{x}^n - \boldsymbol{\mu}(\mathbf{x}^n)\|^2$$

1. **Başlatma:** Birbirinden iyi ayrılmış merkezler $\boldsymbol{\mu}_j$ seçin.
2. **Güncelle:** $\mathcal{S}_j$ 'yi, $\boldsymbol{\mu}_j$ 'ye en yakın tüm noktalar olacak şekilde güncelleyin:

$$\mathcal{S}_j \leftarrow \left\{ \mathbf{x}^n : \|\mathbf{x}^n - \boldsymbol{\mu}_j\| \leq \|\mathbf{x}^n - \boldsymbol{\mu}_l\| \text{ for } l = 1, \dots, k \right\}$$

3. $\boldsymbol{\mu}_j$ 'yi $\mathcal{S}_j$ kümesinin ağırlık merkezi (centroid) olacak şekilde güncelleyin:

$$\boldsymbol{\mu}_j \leftarrow \frac{1}{|\mathcal{S}_j|} \sum_{\mathbf{x}^n \in \mathcal{S}_j} \mathbf{x}^n$$

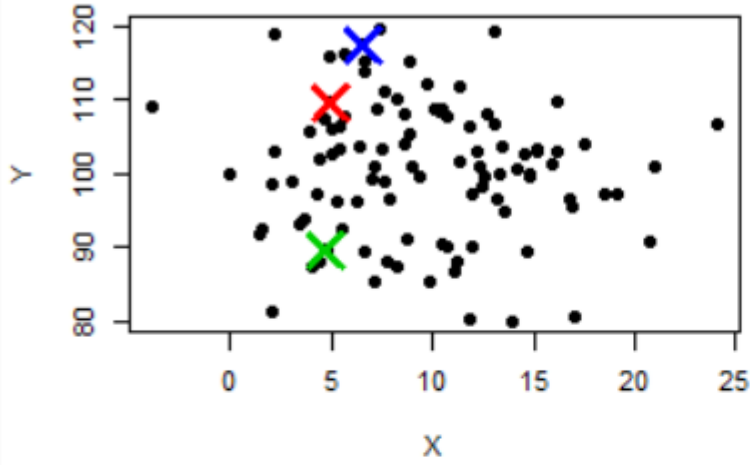
4. E_{in} istenilen azalmayı sağlayana kadar 2 ve 3. adımları tekrarlayın.

Notlar:

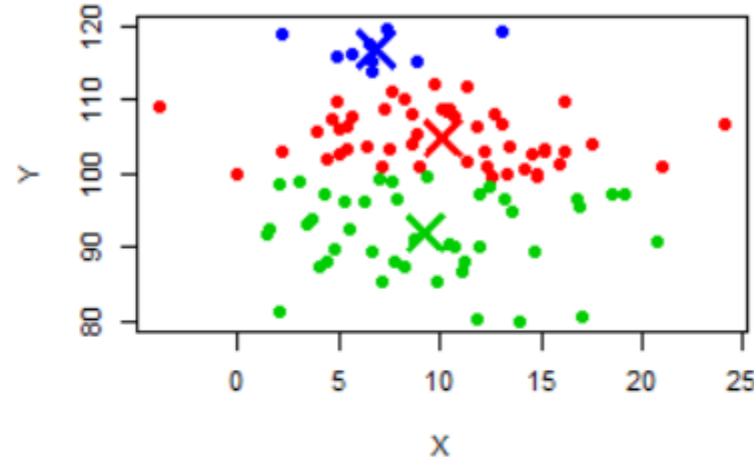
- Algoritmanın her adımında hata sürekli azalır.
- Hata sonsuza kadar sürekli azalamaz; çünkü yalnızca sonlu sayıda kümeleme (olası kümeleme biçimi) vardır.
- Dolayısıyla, algoritma sonlu bir sürede yakınsar.
- Üstel sayıda (exponential) olası kümeleme bulunduğundan, yakınsama uzun zaman alabilir.
- Ayrıca, algoritmanın ardışık (alternatifleyen) yapısı nedeniyle yakınsama en iyi (optimal) çözüme olmayabilir.
- Algoritma, **pratikte genel olarak çok iyi çalışır.**

k-ortalama (k-means) için örnek

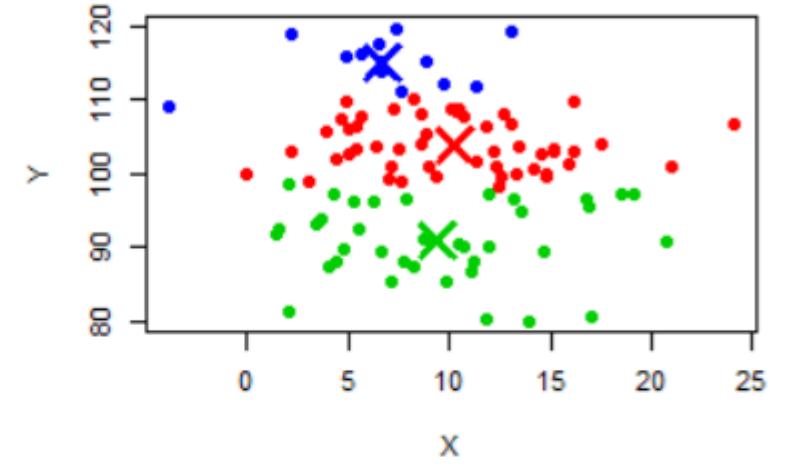
Iteration 1



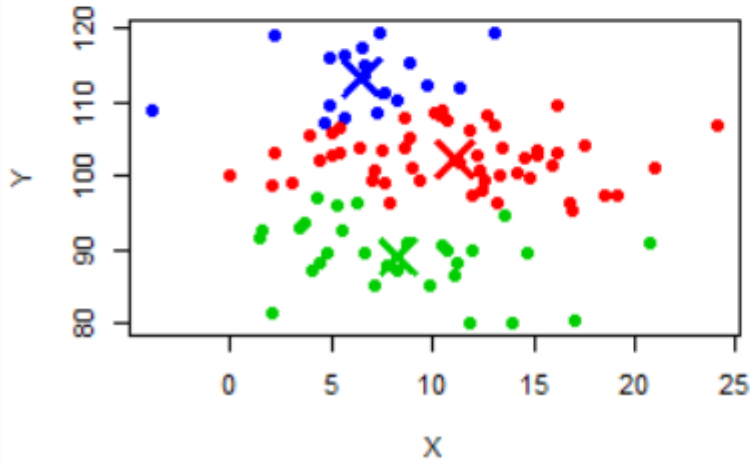
Iteration 2



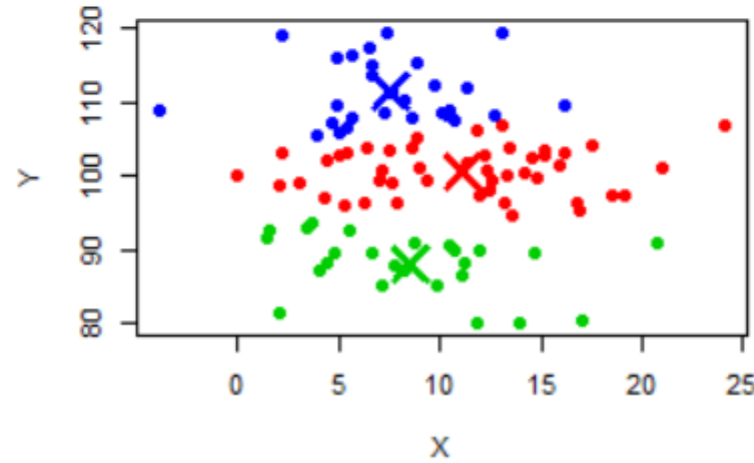
Iteration 3



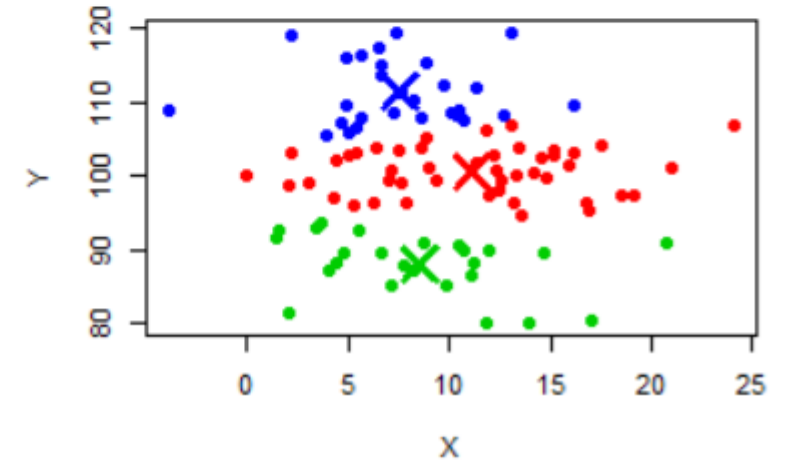
Iteration 6



Iteration 9



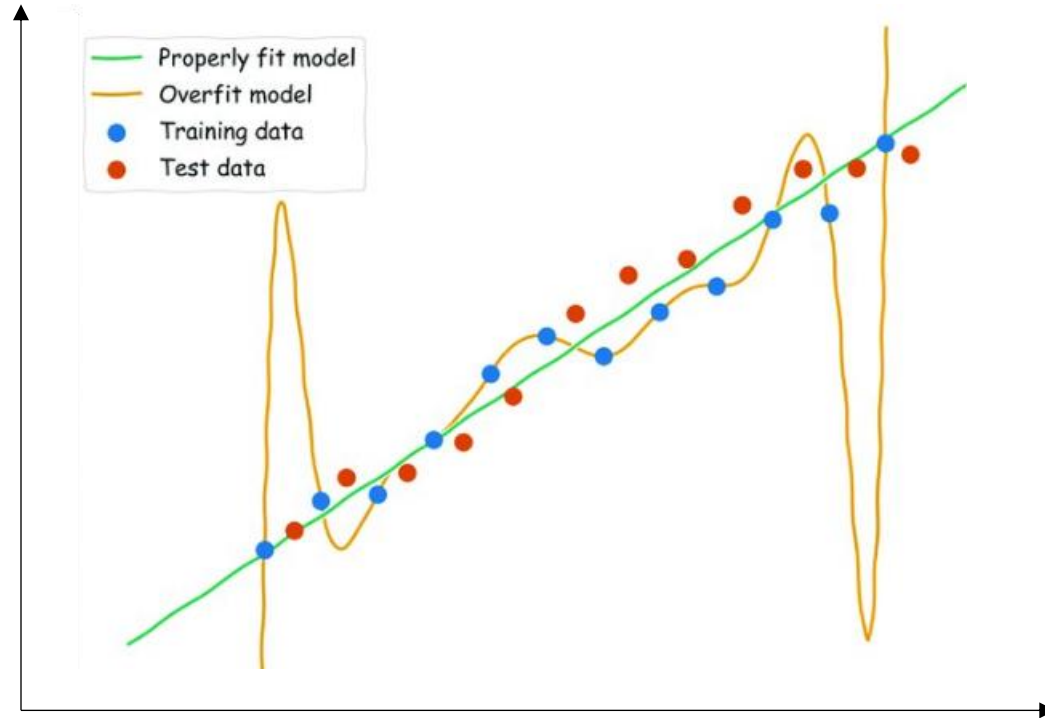
Converged!



Regresyonda regülarizasyon (Regularized regression)

Düzenlileştirme (regularization) nedir?

Unutmayın ki “**çok esnek modeller**” (örneğin derecesi yüksek bir polinom), genel olarak aşırı uyuma (overfitting) yol açar.



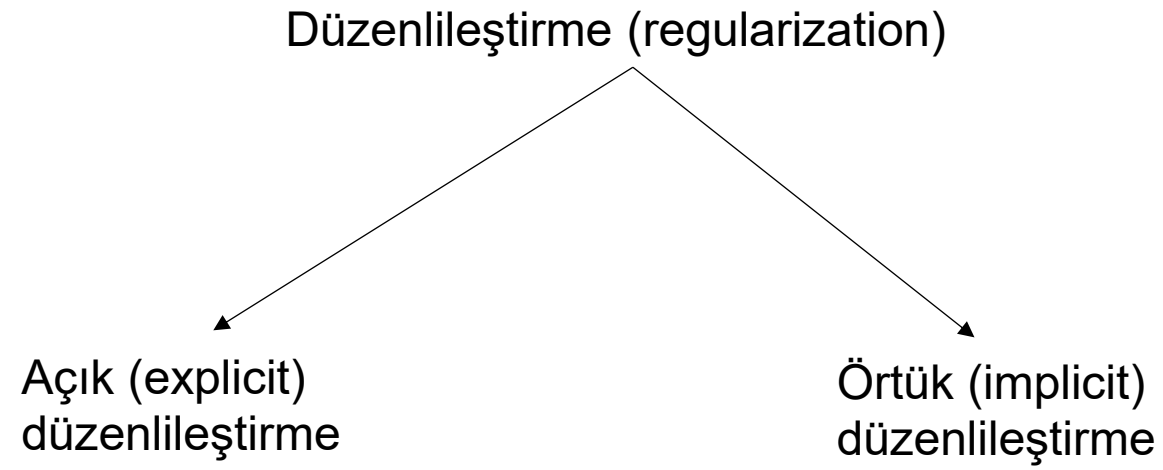
Düzenlileştirme (regularization), **aşırı uyumu** (overfitting) **ortadan kaldırmak ya da azaltmak** için model esnekliğini sınırlayan bir yöntemdir.

- Düzenlileştirme (regularization), hem parametrik hem de parametrik olmayan (non-parametric) modellere uygulanabilir.
- Ancak bu derste odağımız **parametrik modeller** üzerinde olacaktır.

- Parametrik bir modelde düzenlileştirmenin amacı, **“veri bizi gerçekten aksi yönde ikna etmedikçe, parametre vektörü w 'yi değerler açısından küçük tutmaktır”**.

veya

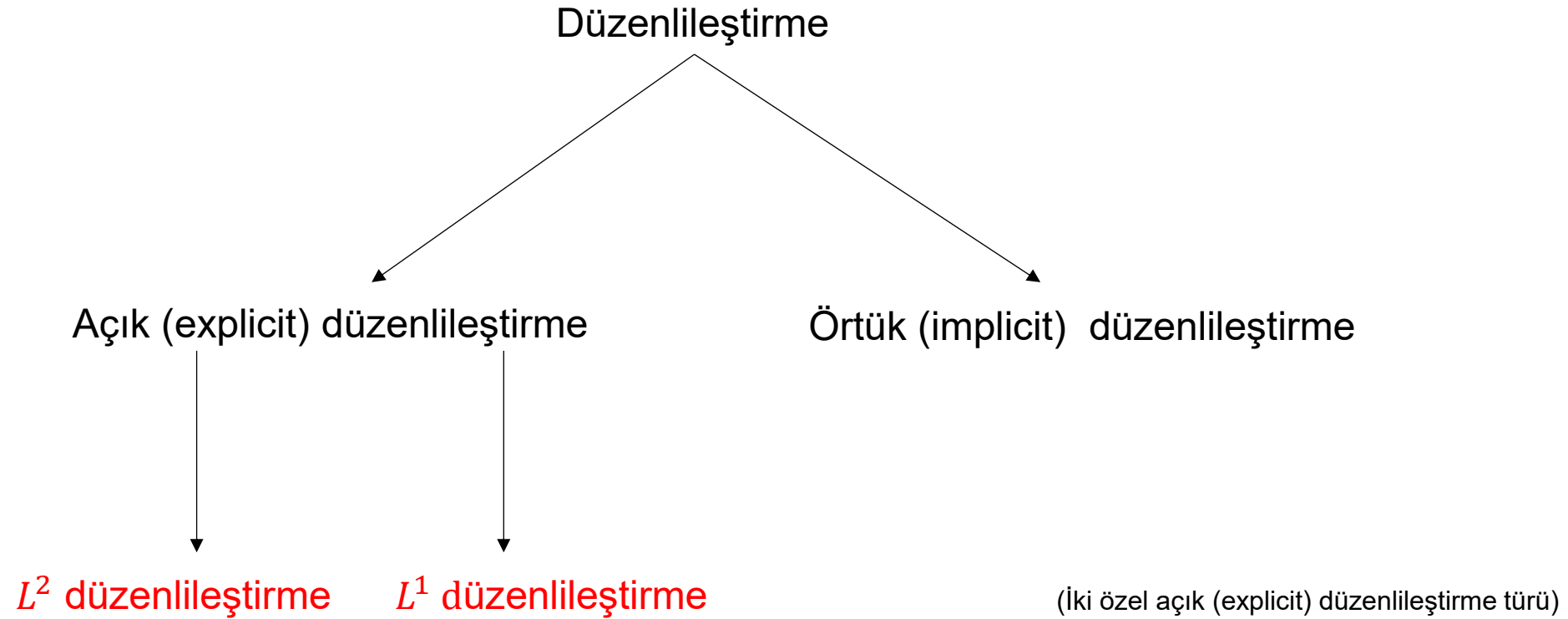
- Alternatif olarak, **“ w içindeki parametrelerin küçük değerlere sahip olduğu bir model, daha büyük parametre değerlerine sahip bir model kadar olmasa bile veriye neredeyse aynı derecede iyi uyuyorsa, küçük parametre değerlerine sahip olan model tercih edilmelidir”**.



$$\hat{\mathbf{w}} = \underset{\mathbf{w}}{\operatorname{argmin}} \underbrace{J(\mathbf{w}; \mathcal{D})}_{(i)} + \underbrace{R(\mathbf{w}; \underbrace{\boldsymbol{\lambda}}_{(iii)})}_{(ii)}$$

Bu ifade üç önemli unsur içerir :

- (i) eğitim verisine iyi uyumu teşvik eden maliyet (cost) fonksiyonu;
- (ii) parametre değerlerinin küçük olmasını teşvik eden düzenlileştirme (regularization) terimi; ve
- (iii) (i) ile (ii) arasındaki ödünleşimi (trade-off) belirleyen düzenlileştirme (regularization) parametre vektörü $\boldsymbol{\lambda}$.



$$J_R(\mathbf{w}) = J(\mathbf{w}; \mathcal{D}) + \boxed{\lambda \|\mathbf{w}\|_2^2}$$

burada $J(\mathbf{w}; \mathcal{D})$, genel bir parametrik makine öğrenmesi modelinin eğitimi için kullanılan özgün maliyet (cost) fonksiyonudur.

L^2 düzenlileştirme **doğrusal regresyona** uygulandığında (bu birleşim genellikle **Ridge Regresyon** olarak bilinir), şu elde edilir:

$$J_{RR}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N (\mathbf{w}^T \mathbf{x}^i - y^i)^2 + \boxed{\lambda \sum_{j=1}^d w_j^2}$$

$$J_R(\mathbf{w}) = J(\mathbf{w}; \mathcal{D}) + \boxed{\lambda \|\mathbf{w}\|_1}$$

burada $J(\mathbf{w}; \mathcal{D})$, genel bir parametrik makine öğrenmesi modelinin eğitimi için kullanılan özgün maliyet (cost) fonksiyonudur.

L^1 düzenlileştirme **doğrusal regresyona** uygulandığında (bu birleşim genellikle **LASSO** olarak bilinir), şu elde edilir:

$$J_{\text{LASSO}}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N (\mathbf{w}^T \mathbf{x}^i - y^i)^2 + \lambda \sum_{j=1}^d |w_j|$$

Not: LASSO, *Least Absolute Shrinkage and Selection Operator* ifadesinin kısaltmasıdır.

$$J_{RR}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N (\mathbf{w}^T \mathbf{x}^i - y^i)^2 + \lambda \sum_{j=1}^d w_j^2$$

Yukarıdaki hedef fonskyonu için aşağıdaki denklemi elde edebiliriz (Nasıl?):

$$(X^T X + N\lambda I_d) \mathbf{w} = X^T \mathbf{y}$$

$\lambda > 0$ için, $X^T X + N\lambda I_d$ matrisi her zaman terslenebilir (invertible) olur ve şunu elde ederiz:

$$\mathbf{w}_{\text{lin}} = (X^T X + N\lambda I_d)^{-1} X^T \mathbf{y}$$

Bu, doğrusal regresyonda düzenleme kullanmanın başka bir nedenini de ortaya koyar; yani $X^T X$ terslenebilir olmadığında.

$$J_{\text{LASSO}}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N (\mathbf{w}^T \mathbf{x}^i - y^i)^2 + \lambda \sum_{j=1}^d |w_j|$$

L^2 düzenlileştirmeli doğrusal regresyonun aksine, LASSO için kapalı biçimli (closed-form) bir çözüm mevcut değildir. Neden olabilir?

Ancak, nümerik optimizasyon kullanılarak problem verimli bir şekilde çözülebilir.

- ❑ L^2 düzenlileştirme, **tüm parametreleri küçük değerlere doğru** iter (ancak mutlaka tam olarak sıfıra değil). Neden?
- ❑ L^1 düzenlileştirme **yalnızca birkaç parametrenin sıfırdan farklı olduğu** ve geri kalanlarının tam olarak sıfır olduğu, seyrek (sparse) çözümleri tercih etme eğilimindedir. Neden?
- ❑ Yanıtları bulamazsanız, şu kaynağa göz atın. : <https://www.youtube.com/watch?v=76B5cMEZA4Y>
- ❑ Dolayısıyla, L^1 düzenlileştirme bazı girdileri (öznitelikleri) etkili biçimde “**devre dışı**” bırakabilir (karşılık gelen parametre w_k 'yi sıfıra ayarlayarak) ve bu nedenle girdi (öznitelik) seçimi yöntemi olarak kullanılabilir.

$$\hat{\mathbf{w}} = \underset{\mathbf{w}}{\operatorname{argmin}} J(\mathbf{w}; \mathcal{D}) + R(\mathbf{w}; \boldsymbol{\lambda})$$

burda

$$R(\mathbf{w}; \boldsymbol{\lambda}) = \lambda_1 \|\mathbf{w}\|_1 + \lambda_2 \|\mathbf{w}\|_2^2$$

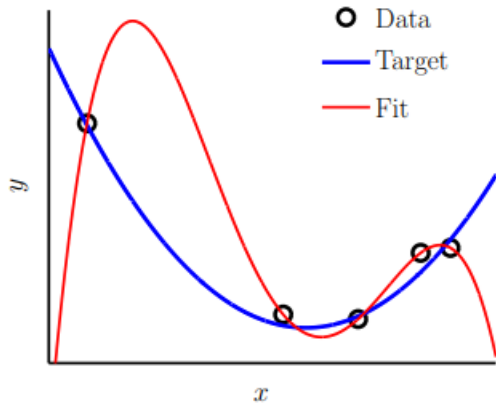
Not: Karışık L^1, L^2 düzenlileştirme, çoğunlukla “**elastic net düzenlileştirme**” olarak adlandırılır.

Farklı düzeylerde düzenleme: bir örnek

Farklı λ değerleriyle $E_{in}(\mathbf{w}) + \lambda \mathbf{w}^T \mathbf{w}$ 'yi en aza indirmeye çalışalım:

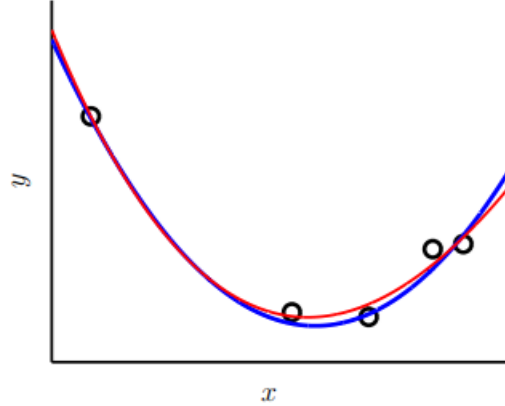
(Veriye yüksek dereceli bir polinom uydurmaya çalışıyoruz.)

$$\lambda = 0$$

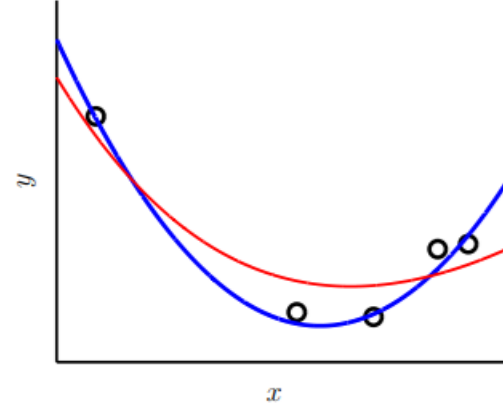


Overfitting
(aşırı uyum)

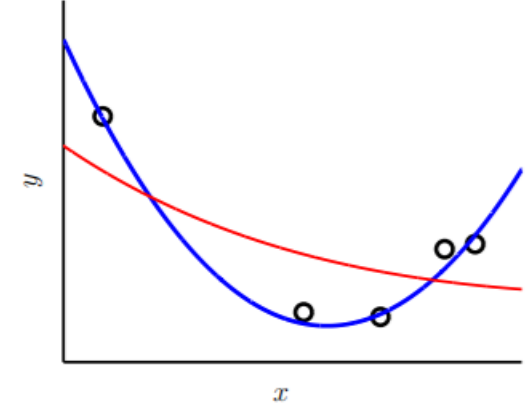
$$\lambda = 0.0001$$



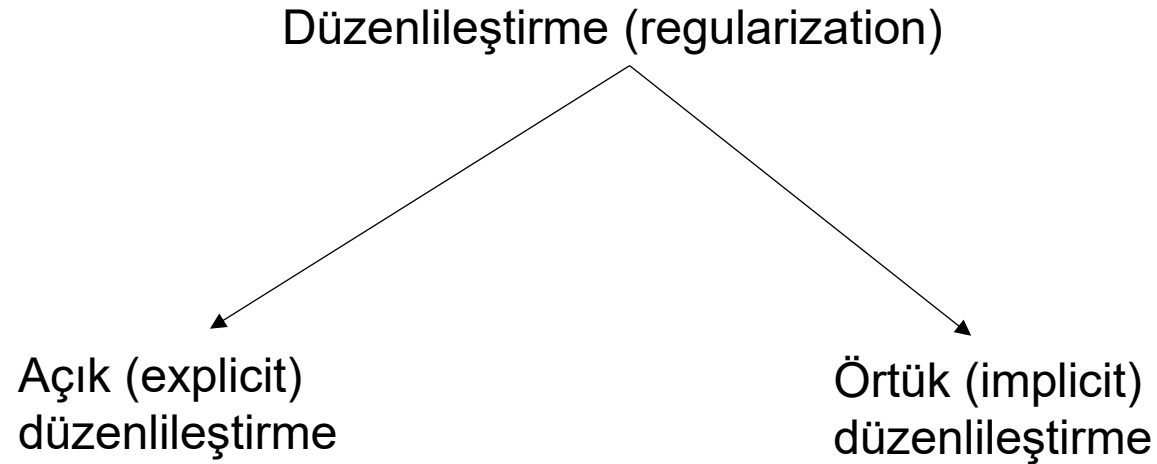
$$\lambda = 0.01$$



$$\lambda = 1$$



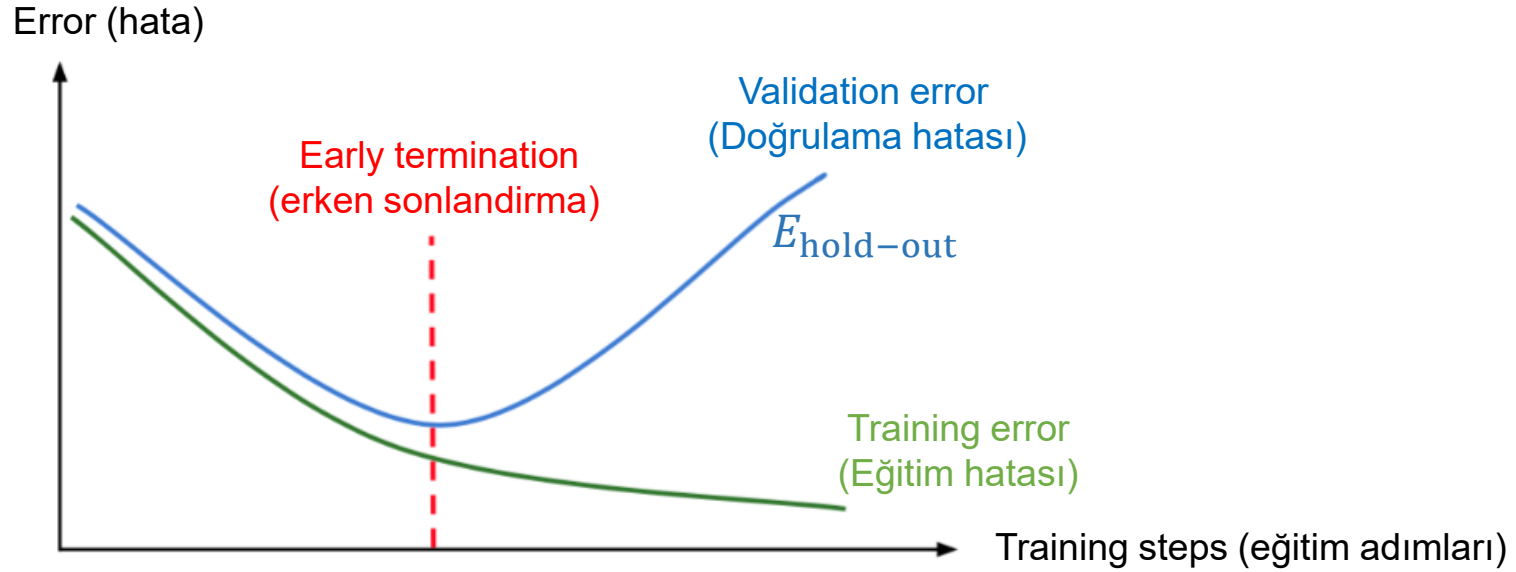
Underfitting
(eksik uyum)



Erken durdurma (early stopping), yinelemeli (iterative) nümerik optimizasyonda maliyet fonksiyonu minimumuna ulaşmadan önce eğitimi erken sonlandırmak anlamına gelir. Örtük (implicit) düzenlileştirme grubundadır.

- Erken durdurma, bir optimizasyon sürecini vaktinden önce sonlandırmak açısından sezgiye aykırı görünebilir; ancak **pratikte yararlı olduğu kanıtlanmıştır**.
- Erken durdurmanın, bazı modellerde—özellikle derin öğrenme modellerinde—aşırı uyumu önlemek açısından **pratikte önemli olduğu gösterilmiştir**.

Erken durdurma nasıl uygulanır?



- ❑ Bir miktar doğrulama verisini (hold-out) kenara ayırıp, her “t” optimizasyon adımından sonra güncel $w(t)$ ağırlıkları için bir $E_{\text{hold-out}}$ değeri hesaplayarak erken durdurma yöntemini uygulayabiliriz.
- ❑ Tipik olarak, $E_{\text{hold-out}}$ başlangıçta azalır; ancak bir süre sonra minimuma ulaşır ve ardından artmaya başlar; eğitim maliyet fonksiyonu ise tekdüze (monotonik) biçimde azalmaya devam eder.
- ❑ Daha sonra optimizasyon, $E_{\text{hold-out}}$ değerinin minimuma ulaştığı noktada sonlandırılır.